

▼ EV Charging Station Waiting Time Prediction

A machine learning approach to predicting and optimizing EV charging wait times using regression models and operational data.

Domain

Electric Vehicle Infrastructure

Method

Supervised Regression Modeling

Stack

Python · scikit-learn · pandas

Author

Abinaya · B.Tech CSE



Overview

The Problem

Electric Vehicle (EV) adoption is growing rapidly, and with it comes increasing pressure on charging infrastructure. Long or unpredictable waiting times at charging stations are a major pain point for EV owners and a planning challenge for station operators.

The Solution

This project uses data analysis and machine learning to **predict waiting times at EV charging stations** based on operational and environmental factors such as charger availability, charging type, vehicle type, weather, traffic, and time of day.

The insights and models produced can help operators optimize charger allocation and help drivers plan their charging visits more efficiently.



Objectives

1

Analyze Key Factors

Identify and analyze the key factors that influence EV charging station waiting times across operational and environmental dimensions.

2

Prepare the Data

Handle outliers and prepare the dataset for modeling through preprocessing, encoding, and feature engineering.

3

Build & Compare Models

Build and compare multiple regression models to predict waiting time with measurable accuracy.

4

Identify Best Model

Identify the best-performing model based on evaluation metrics including R^2 Score and Mean Squared Error.

Dataset

File: ev_charging_dataset_with_added_parameters.xlsx

Column	Description
Charging_Type	Fast or Normal charging
Available_Chargers	Number of chargers available at the station
Vehicle_Type	Type of EV (e.g., car, bike, bus)
City	Location of the charging station
Time_Slot	Time period of charging session
Weather	Weather conditions during the session
Holiday	Whether the day was a holiday
Traffic_Level	Traffic conditions near the station
Waiting_Time	Target variable — time (in minutes) a user waited to charge

Tech Stack



Language

Python



Data Handling

pandas · numpy



Visualization

matplotlib · seaborn



Modeling

scikit-learn



Environment

Jupyter Notebook / Google Colab



Project Workflow

01

Import Libraries

Load pandas, numpy, matplotlib, seaborn, and scikit-learn

02

Load Dataset

Read the Excel dataset into a pandas DataFrame

03

Exploratory Data Analysis (EDA)

Check shape, structure, summary statistics, and missing values

04

Data Visualization

Boxplot of Waiting_Time, countplot of Vehicle_Type, bar plots for Charging_Type and Available_Chargers

05

Data Preprocessing

Cap outliers, encode Charging_Type, drop non-predictive columns, one-hot encode categoricals

06

Correlation Analysis

Heatmap to identify variables most influencing waiting time

07

Train-Test Split

80% training / 20% testing using `train_test_split`

08

Model Building & Evaluation

Train and compare three regression models

09

Results

Compare actual vs. predicted waiting times for the final model

Data Preprocessing

Outlier Capping

Cap extreme outliers in `Waiting_Time` — values above **500** are capped at **500** to reduce skewing effect on model training.

Numeric Encoding

Encode `Charging_Type` numerically: **Fast = 1, Normal = 0** to make it directly usable by regression models.

Drop Non-Predictive Columns

Remove `Station_ID` and `Date` — these are identifiers and timestamps that do not contribute predictive signal to the model.

One-Hot Encoding

One-hot encode all remaining categorical features: `City`, `Time_Slot`, `Vehicle_Type`, `Weather`, `Holiday`, and `Traffic_Level`.



Model Building & Evaluation

Three regression models are trained and compared. Each model is evaluated using **R² Score** (proportion of variance explained) and **MSE** (mean squared error).

Model	Description
Linear Regression	Baseline model capturing linear relationships between features and waiting time. Provides a performance floor for comparison.
Ridge Regression	Regularized linear model; alpha tuned via GridSearchCV to penalize large coefficients and reduce overfitting.
Random Forest Regressor	Ensemble model tuned via GridSearchCV (n_estimators, max_depth, criterion) to capture non-linear patterns in the data.



Results Summary

 Fill in with your final output values after running the notebook.

Model	R ² Score	MSE
Linear Regression	0.69	0.4353
Ridge Regression	0.69	0.4353
Random Forest Regressor	0.99	10.78

R² Score

Measures the proportion of variance in waiting time explained by the model. A score closer to **1.0** indicates a better fit. Compare across models to identify which captures the most signal.

Mean Squared Error (MSE)

Measures the average squared difference between actual and predicted waiting times. A **lower MSE** indicates more accurate predictions. Use alongside R² for a complete picture of model performance.



How to Run

1 Clone or Download the Repository

Obtain the project files from the repository to your local machine or Google Colab environment.

2 Install Required Dependencies

Run the following command to install all necessary libraries:

```
pip install pandas numpy matplotlib seaborn scikit-learn  
openpyxl
```

3 Place the Dataset

Place `ev_charging_dataset_with_added_parameters.xlsx` in the project directory. Update the file path in the notebook if not using Google Colab's default `/content/` path.

4 Run the Notebook

Open the notebook and run all cells sequentially to reproduce the full analysis, preprocessing, model training, and results.

Key Takeaways

Problem Scope

Predicting EV charging wait times addresses a real and growing infrastructure challenge as EV adoption accelerates globally.

Feature Richness

The dataset captures operational, environmental, and temporal factors — charger availability, weather, traffic, time slot, and vehicle type — enabling multi-dimensional modeling.

Model Comparison

Three models — Linear Regression, Ridge Regression, and Random Forest — are benchmarked to identify the best fit for this prediction task.

Practical Impact

Accurate predictions can help station operators optimize charger allocation and help drivers plan charging visits more efficiently.

Results table to be completed after running the notebook. The Random Forest Regressor, with its ability to capture non-linear patterns and hyperparameter tuning via GridSearchCV, is expected to outperform the linear baselines on both R^2 and MSE.